CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

CASE STUDY
ON
YOUTUBE DATA ANALYSIS AND VISUALIZATION USING PYTHON

PYTHON PROGRAMMING

25CAH-221

SUBMITTED BY

Name: S. S. Sahil

UID: 25BCD10109

Branch: BCA (Data Science)

Section: 25BCD2(B)

Semester: 3

SUBMITTED TO

Name: Ms. PREETI

Designation: Professor

Date of Submission: 08/10/2026

Signature: _____

Abstract

This case study analyses a small sample of YouTube video data to understand how audience engagement differs across video categories. The dataset contains ten video records divided equally among five categories: Music, Gaming, Education, Tech and Comedy. For every video, the number of views, likes and comments is recorded. The work is carried out in Python using NumPy, Pandas, Matplotlib and Seaborn.

The study calculates basic statistics such as the mean, median and standard deviation, groups the data by category, sorts and filters the records, and measures the correlation between views, likes and comments. Ten different visualizations are then used to present the data. In this sample, the average number of views is 136,000, Tech videos have the highest category average (225,000) and Music videos have the lowest (70,000). Views, likes and comments move together perfectly in the sample. The dataset is small and illustrative, so the results describe only the sample used here and must not be treated as statistics for YouTube as a whole.

1. Introduction

YouTube is one of the most widely used video platforms in the world, and creators and marketers regularly study how viewers respond to different kinds of videos. The three most visible signals of that response are views, likes and comments. Views show how many times a video was watched, likes show approval, and comments show that viewers cared enough to join a discussion.

Data analysis helps turn such numbers into clear observations. Python has become a popular choice for this work because it offers dedicated libraries for numerical computing, data handling and plotting. This case study applies those libraries to a sample of YouTube video data and explains each step in simple terms, from creating the dataset to interpreting ten different graphs.

2. Background of the Study

Online video content is divided into categories such as music, gaming, education, technology and comedy, and each category attracts a different kind of audience. Understanding which categories draw more views, and whether higher views also bring more likes and comments, is a common question in digital media analysis.

As part of the Python Programming course, this project gives practice in the standard workflow of a data analyst: preparing data, checking its quality, calculating summary statistics, and presenting the results visually. A small, controlled dataset was chosen so that every number can be checked by hand and every graph can be understood easily. The project therefore focuses on learning the methods rather than on collecting large-scale platform data.

3. Problem Statement

Raw figures for views, likes and comments are difficult to compare by reading them as a list. It is not obvious at a glance which category performs best, how widely the values are spread, or whether the three measures are related. The problem addressed in this case study is how to use Python to summarize and visualize YouTube video data so that differences between categories and relationships between engagement measures become easy to see and explain.

4. Objectives of the Study

The main objectives of this case study are listed below.

- To create a structured dataset of YouTube videos using NumPy arrays and a Pandas DataFrame.
- To check the dataset for missing values, duplicate records and correct data types.
- To calculate the mean, median and standard deviation of the numerical columns.
- To compare video categories by grouping the data and calculating average views.
- To sort and filter the records to find the most-viewed videos.
- To measure the relationship between views, likes and comments using correlation.
- To present the data through ten different visualizations using Matplotlib and Seaborn and to interpret each one.

5. Scope of the Study

The study covers the analysis of one small sample dataset of ten video records and four fields: category, views, likes and comments. The analysis is descriptive, which means it summarizes and displays the data that is already available. It does not collect live data, does not use the YouTube API, and does not build any machine learning or prediction model.

Because the dataset has only two videos per category, the conclusions apply to this sample only. They show how the analysis methods work, and they should not be read as facts about the real YouTube platform.

6. Dataset Description

The dataset is created directly in the Python program using four NumPy arrays. It contains ten records, with exactly two videos for each of the five categories. The values are used exactly as written in the source code. Table 1 shows the complete dataset, and Table 2 describes its columns.

Table 1: Dataset used in the study

No.	Category	Views	Likes	Comments
1	Music	50,000	5,000	500
2	Gaming	80,000	8,000	800
3	Education	120,000	12,000	1,200
4	Tech	200,000	20,000	2,000
5	Comedy	150,000	15,000	1,500
6	Music	90,000	9,000	900
7	Gaming	180,000	18,000	1,800
8	Education	100,000	10,000	1,000
9	Tech	250,000	25,000	2,500
10	Comedy	140,000	14,000	1,400

Table 2: Description of dataset columns

Column	Type	Meaning
Category	Text (categorical)	The type of video: Music, Gaming, Education, Tech or Comedy
Views	Integer	Number of times the video was viewed
Likes	Integer	Number of likes received by the video
Comments	Integer	Number of comments posted on the video

The data is sample data prepared for this project. It was not downloaded from YouTube or collected through the YouTube API.

7. Tools and Technologies Used

The entire project is written in Python. Four libraries are used, each for a specific purpose, as summarized in Table 3.

Table 3: Tools and libraries used

Tool	Purpose in this project
Python	General-purpose programming language used to write the whole program
NumPy	Stores the data in arrays and calculates the mean, median, standard deviation, maximum, unique categories and correlation matrix
Pandas	Creates the DataFrame; supports missing-value, duplicate and data-type checks, grouping, sorting, filtering and correlation; builds the data frame used for the pair plot
Matplotlib	Draws the line chart, bar chart, histogram, pie chart and scatter plot
Seaborn	Draws the box plot, violin plot, heatmap, count plot and pair plot

8. Methodology

The work followed a simple sequence of steps, which is the usual workflow of a small data analysis project.

1. **Data creation.** Four NumPy arrays (categories, views, likes and comments) were written in the program, each holding ten values.
2. **DataFrame preparation.** The arrays were combined into a Pandas DataFrame with the columns Category, Views, Likes and Comments.
3. **Data checking.** The DataFrame was checked for missing values, duplicate rows and data types.
4. **Statistical analysis.** The mean, median and standard deviation were calculated, followed by category-wise averages, sorting, filtering and correlation.
5. **Visualization.** Ten graphs were produced, five with Matplotlib and five with Seaborn.
6. **Interpretation.** Each result and graph was read carefully and the findings were written down.

The project is kept in two script versions that are both reproduced in the appendix. The main analysis and visualization program (Appendix A) calculates the averages, the category-wise averages and the correlation matrix and draws all ten graphs. The analysis section of the project report script (Appendix B) additionally contains the Pandas data-checking, median, standard

deviation, grouping, sorting, filtering and correlation statements. Where an operation is described in this case study, the script that contains it is named.

9. Data Cleaning and Preprocessing

Before any analysis, it is good practice to check whether the data is complete and in a usable form. Three checks are used in this project (Appendix B). They are performed on the Pandas DataFrame created from the four arrays.

9.1 DataFrame Creation

A DataFrame is a table with rows and labelled columns, similar to a spreadsheet. The program builds it by passing a dictionary of the four arrays to pd.DataFrame(). This turns separate lists of values into one organized structure on which grouping, sorting and filtering can be applied easily.

9.2 Missing Value Check

The expression df.isnull().sum() counts the empty entries in each column. Missing values can distort averages and cause errors in calculations, so finding them early is important. For this dataset the count is zero for all four columns, which means every record is complete.

9.3 Duplicate Record Check

The expression df.duplicated().sum() counts rows that are exact copies of an earlier row. Duplicates would count the same video twice and bias the results. The result is zero, so no duplicate records exist. Note that the two videos in each category have different values and are therefore separate records.

9.4 Data Type Inspection

The df.dtypes attribute shows the data type of each column. Views, Likes and Comments are stored as integers (int64), which is suitable for counting data and for mathematical operations. The Category column holds text. Correct data types matter because a number stored as text cannot be averaged or plotted properly. As the data was clean, no cleaning action such as removing rows or converting types was required.

10. Exploratory Data Analysis

Exploratory data analysis is the first look at the data to understand its size, structure and general pattern. In this study the first look shows the following.

- The dataset has 10 records and 4 columns, with no missing or duplicate values.
- There are five categories and each one has exactly two videos, so the sample is balanced.
- Views range from 50,000 (a Music video) to 250,000 (a Tech video), and comments range from 500 to 2,500.
- In every record, likes are exactly 10% of views and comments are exactly 1% of views. This fixed proportion explains the perfect correlation seen later, and it also shows that the data is illustrative rather than real.

The main program prints the total number of videos (len(views)), the three averages and the highest views (np.max), which gives this overview in a few lines. The graphs in Section 12 then explore the same data visually.

11. Statistical Analysis

This section explains each operation, its purpose and the result obtained. Table 4 gives a summary of the numerical results.

11.1 Mean

The mean (average) is the sum of all values divided by the number of values, calculated with np.mean(). It gives the typical level of a column. The mean views are 136,000, the mean likes are 13,600 and the mean comments are 1,360. These values come from the main program (Appendix A).

11.2 Median

The median is the middle value when the data is arranged in order, calculated with np.median(). With ten values it is the average of the fifth and sixth values. The median is not affected by extreme values, so comparing it with the mean shows whether a few very large values are pulling the average upward. The medians are 130,000 for views, 13,000 for likes and 1,300 for comments. Each median is slightly below its mean, which indicates that the highest values (the Tech videos) raise the average a little.

11.3 Standard Deviation

The standard deviation measures how far the values are spread around the mean. A small value means the data is close to the average, while a large value means it is widely spread. The program uses np.std(views), which calculates the population standard deviation and gives

approximately 57,827 views. This is a large spread compared with the mean of 136,000, showing that the videos differ considerably in their number of views.

11.4 Grouping and Aggregation

Grouping splits the data into categories, and aggregation calculates one summary value for each group. The report script does this with df.groupby("Category")["Views"].mean(). The main program obtains the same result with a for loop that selects the views of each category using the condition categories == category and then applies np.mean(). Grouping is the key step for comparing categories. Table 5 shows the result.

11.5 Sorting

Sorting arranges the records in order. The report script uses df.sort_values("Views", ascending=False) to list the videos from the highest to the lowest number of views. The three most-viewed videos are the Tech video with 250,000 views, the other Tech video with 200,000 views and a Gaming video with 180,000 views.

11.6 Filtering

Filtering selects only the rows that satisfy a condition. The expression df[df["Views"] > 150000] returns the videos with more than 150,000 views. Three videos meet this condition: both Tech videos (200,000 and 250,000 views) and one Gaming video (180,000 views). The Comedy video with exactly 150,000 views is not included because the condition is strictly greater than.

11.7 Correlation Analysis

Correlation measures how strongly two variables move together on a scale from -1 to +1. A value near +1 means that when one variable rises, the other also rises. The report script calculates it with df[["Views", "Likes", "Comments"]].corr(), and the main program uses np.corrcoef(). Both give a correlation of 1.00 between every pair of the three columns. The relationship is therefore a perfect positive one in this sample. Because the values follow fixed proportions (see Section 10), this result is expected here; real YouTube data would rarely be this perfect.

Table 4: Summary of statistical results

Statistic	Views	Likes	Comments
Total videos	10	10	10
Mean	136,000	13,600	1,360
Median	130,000	13,000	1,300
Minimum	50,000	5,000	500
Maximum	250,000	25,000	2,500
Standard deviation (views)	57,827	-	-

Table 5: Average views by category

Category	Video 1 views	Video 2 views	Average views
Comedy	150,000	140,000	145,000
Education	120,000	100,000	110,000
Gaming	80,000	180,000	130,000
Music	50,000	90,000	70,000
Tech	200,000	250,000	225,000

12. Data Visualization and Interpretation

Ten visualizations were created from the data, five with Matplotlib and five with Seaborn. Matplotlib is the basic plotting library of Python and gives full control over simple charts. Seaborn is built on top of Matplotlib and makes statistical graphs such as box plots and heatmaps easier to draw. The figures below were generated by running the program in Appendix A.

12.1 Line Chart (Matplotlib)

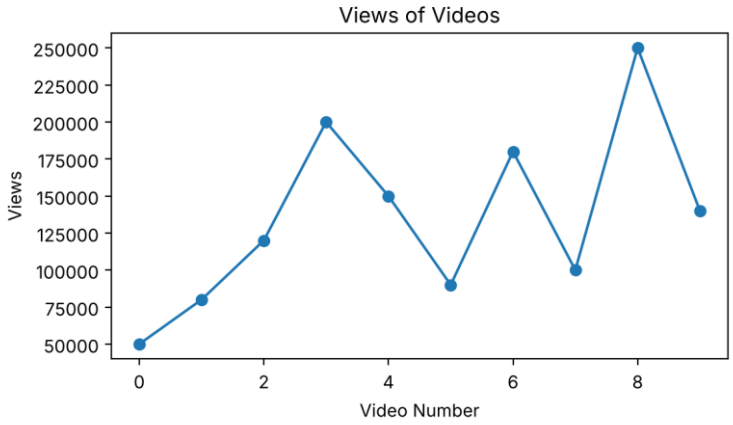


Figure 1: Line chart of views of the ten videos

What it represents. The line chart plots the views of the ten videos in the order in which they appear in the dataset. The horizontal axis is the video number (0 to 9) and the vertical axis is the number of views. It is drawn with plt.plot() using circular markers.

Why it is useful. A line chart is useful for seeing how values rise and fall from one record to the next and for spotting the highest and lowest points quickly.

Findings in this dataset. The lowest point is the first video (50,000 views) and the highest is the ninth video (250,000 views). The line moves up and down repeatedly, so views do not follow a steady pattern. The video number only shows the position in the dataset and does not represent time, so the chart should not be read as a trend over days or months.

12.2 Bar Chart (Matplotlib)

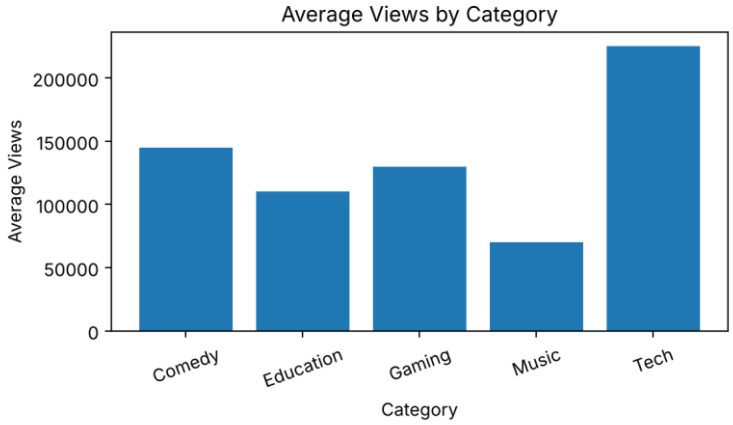


Figure 2: Bar chart of average views by category

What it represents. Each bar shows the average number of views for one category. The averages are calculated in the program with a loop and np.mean() and drawn with plt.bar().

Why it is useful. A bar chart is the best way to compare a single measure across separate categories because the height of each bar can be compared directly.

Findings in this dataset. Tech has the tallest bar (225,000 views) and Music the shortest (70,000 views). Comedy (145,000), Gaming (130,000) and Education (110,000) lie in between. The Tech average is more than three times the Music average in this sample.

12.3 Histogram (Matplotlib)

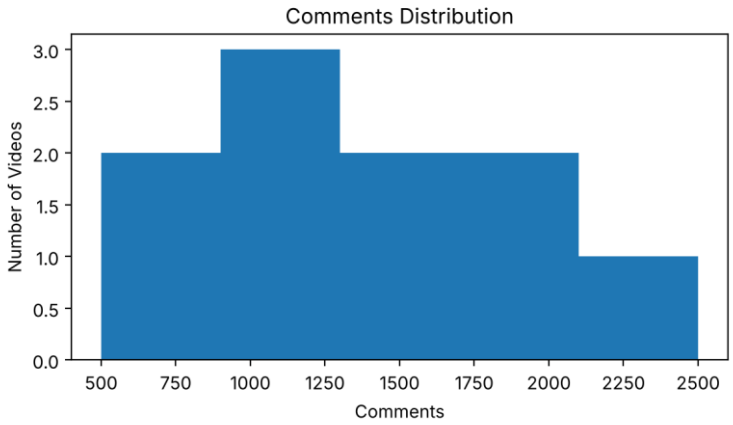


Figure 3: Histogram of the number of comments

What it represents. The histogram divides the range of comment counts into five equal intervals (bins) and shows how many videos fall into each. It is drawn with plt.hist(comments, bins=5).

Why it is useful. A histogram shows the shape of a distribution: where values are concentrated, how far they are spread and whether there are unusual values.

Findings in this dataset. The five intervals of width 400 contain 2, 3, 2, 2 and 1 videos respectively. The interval from 900 to 1,300 comments is the most common one with three videos, and the interval from 2,100 to 2,500 has only one video (2,500 comments). The counts decrease slightly toward higher comment values.

12.4 Pie Chart (Matplotlib)

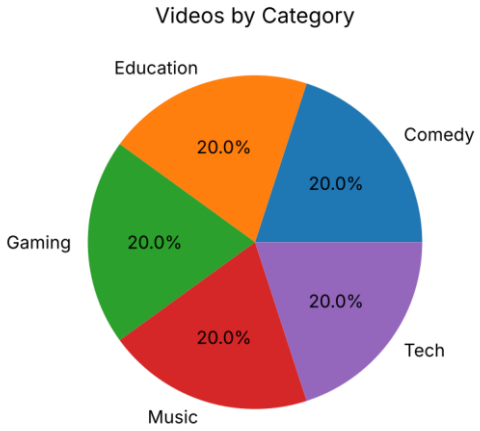


Figure 4: Pie chart of videos by category

What it represents. The pie chart shows each category as a slice, with its size representing the share of the total number of videos. The counts come from np.unique(categories, return_counts=True), and percentages are shown with autopct.

Why it is useful. A pie chart is suitable for showing how a whole is divided into parts when there are only a few categories.

Findings in this dataset. All five slices are equal and each shows 20.0%, because every category has two of the ten videos. The chart confirms that the sample is balanced and that no category is over-represented in the averages.

12.5 Scatter Plot (Matplotlib)

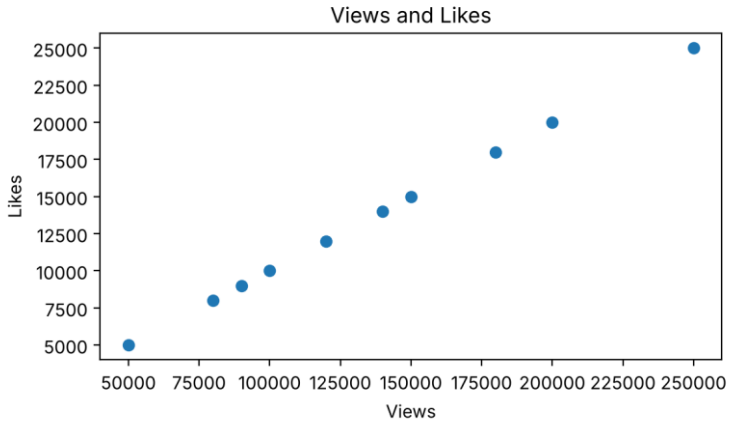


Figure 5: Scatter plot of views and likes

What it represents. Each point is one video, with views on the horizontal axis and likes on the vertical axis. It is drawn with plt.scatter().

Why it is useful. A scatter plot shows whether two numerical variables are related and what the relationship looks like, such as rising, falling or no pattern.

Findings in this dataset. The ten points lie exactly on a rising straight line. Videos with more views have proportionally more likes, which matches the correlation of 1.00. The line reflects the fixed ratio of likes to views (10%) in the sample.

12.6 Box Plot (Seaborn)

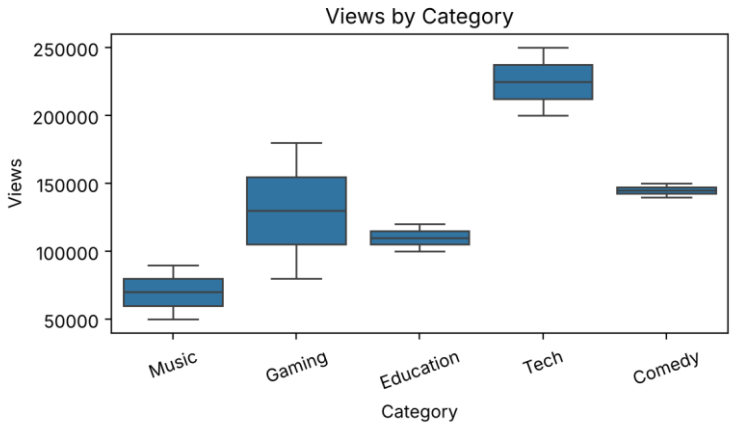


Figure 6: Box plot of views by category

What it represents. For each category, the box shows the middle half of the views (from the first quartile to the third quartile), the line inside the box is the median and the whiskers extend to the smallest and largest values. It is drawn with sns.boxplot().

Why it is useful. A box plot summarizes spread and centre in a compact form and makes it easy to compare several groups side by side.

Findings in this dataset. The Tech box is the highest on the axis, with views between 200,000 and 250,000. Gaming has the widest box because its two videos differ greatly (80,000 and 180,000). Comedy and Education have narrow boxes, showing that their two videos have similar views. Each category has only two points, so the quartiles are calculated from just two values and should be treated as a rough guide.

12.7 Violin Plot (Seaborn)

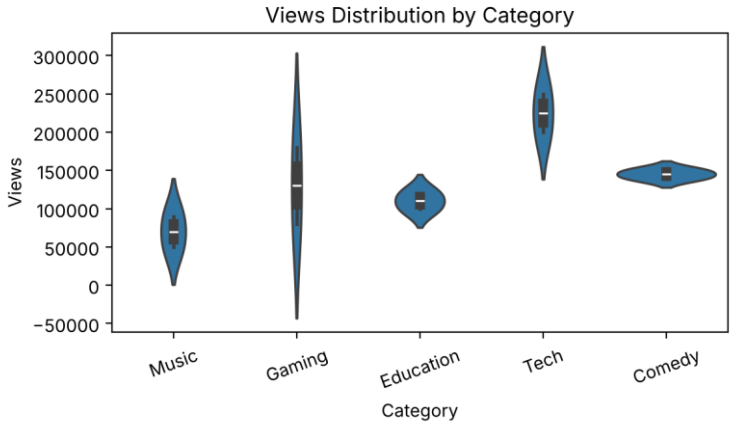


Figure 7: Violin plot of views distribution by category

What it represents. The violin plot combines a box plot with a smooth curve that shows how the values are distributed. A wider part of the shape means more values around that level. It is drawn with sns.violinplot().

Why it is useful. It shows the shape of the distribution of each group, which a box plot alone cannot show.

Findings in this dataset. Tech is placed highest, Music lowest, and Gaming has the longest and thinnest shape because of its large gap between the two videos. Comedy is flat and narrow because its values are close. With only two values per category the smooth curves are estimates, and some shapes extend beyond the actual minimum and maximum values (even below zero for Gaming), so the plot is useful here mainly for understanding the method.

12.8 Heatmap (Seaborn)

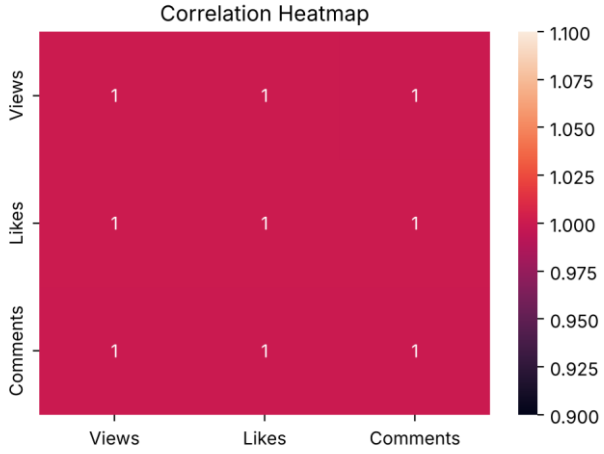


Figure 8: Correlation heatmap of views, likes and comments

What it represents. The heatmap displays the correlation matrix of views, likes and comments as a coloured grid, with the value written in each cell. The matrix is calculated with np.corrcoef() and drawn with sns.heatmap(annot=True).

Why it is useful. A heatmap lets the reader see the strength of several relationships at once, using colour to show how high or low each value is.

Findings in this dataset. Every cell contains the value 1 (a correlation of 1.00), so views, likes and comments are perfectly positively correlated in this sample. The colour is uniform because all nine values are the same. In a real dataset the cells would normally show different values.

12.9 Count Plot (Seaborn)

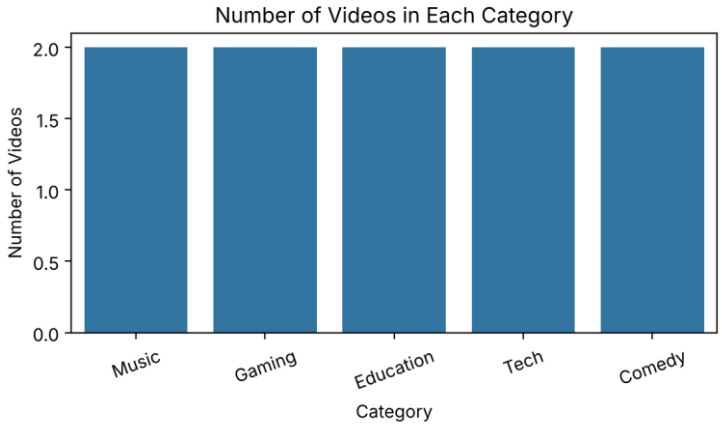


Figure 9: Count plot of the number of videos in each category

What it represents. A count plot shows the number of records in each category as a bar. It is drawn with sns.countplot(), which counts the entries itself.

Why it is useful. It is used to check whether the categories are balanced and how many observations each group contains.

Findings in this dataset. All five bars have the same height of 2, so each category has two videos. This agrees with the pie chart and confirms that the category averages are each based on two records.

12.10 Pair Plot (Seaborn)

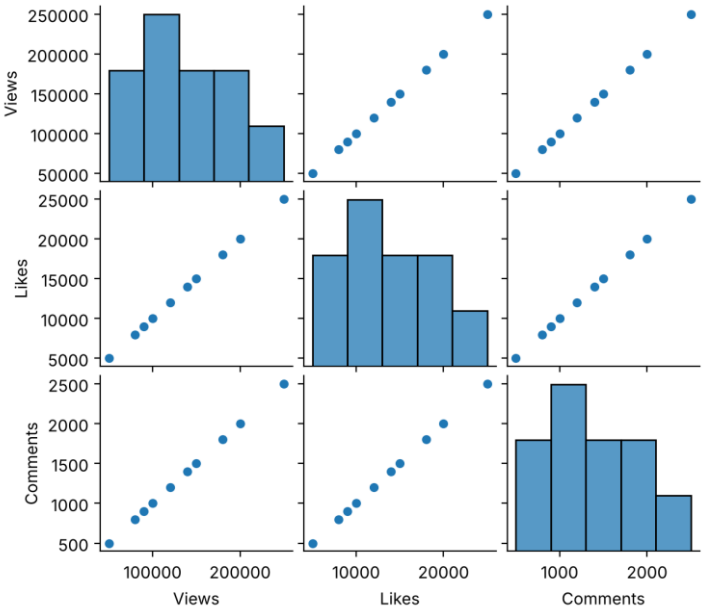


Figure 10: Pair plot of views, likes and comments

What it represents. The pair plot creates a grid of graphs for every pair of numerical columns. The off-diagonal cells are scatter plots of one variable against another, and the diagonal cells are histograms of each variable. It is drawn with sns.pairplot() on a DataFrame of the three columns.

Why it is useful. It gives a quick overview of all relationships and distributions in a dataset using a single command.

Findings in this dataset. All six scatter plots show points on straight rising lines, which confirms the perfect positive relationship between views, likes and comments. The diagonal histograms show how each variable is distributed across its range.

13. Results and Findings

The main results of the analysis are listed in Table 6 and explained below. All results apply only to the ten-record sample used in this project.

Table 6: Key findings

Finding	Result
Total videos	10
Average views / likes / comments	136,000 / 13,600 / 1,360
Median views / likes / comments	130,000 / 13,000 / 1,300
Highest average views	Tech - 225,000
Lowest average views	Music - 70,000
Videos per category	2 in every category
Relationship of views, likes, comments	Perfect positive correlation (1.00)

- **Average level.** The average video in the sample has 136,000 views, 13,600 likes and 1,360 comments. The medians are slightly lower, so a few high-view videos pull the averages up.
- **Category comparison.** Tech has the highest average views (225,000) and Music has the lowest (70,000). Both Tech videos are among the top three in views, while both Music videos are among the four lowest.
- **Balanced sample.** Every category has two videos, so no category has more weight than another.
- **Relationship between measures.** Views, likes and comments rise together. The correlation is 1.00 because likes are exactly 10% and comments exactly 1% of views for every video in this sample.

These findings describe the sample data only. The dataset is too small, and it was prepared for learning, so it cannot support statements about all YouTube videos or about how Tech or Music content performs on the real platform.

14. Challenges and Limitations

- **Very small sample.** With only ten records, one video can change an average noticeably. For example, each category average depends on just two values.
- **Sample data.** The data was not collected from YouTube or its API, so it cannot represent real viewer behaviour.
- **Perfect correlation.** The exact ratios between views, likes and comments make the correlation 1.00, which is unusual in real data and limits what the correlation results can teach.

- **Few values per group.** Box plots and violin plots are designed for larger groups. With two values per category the quartiles and smooth curves are only rough estimates.
- **No time information.** The dataset has no dates, so the line chart shows dataset order and not a trend over time.
- **Limited measures.** Only views, likes, comments and category are available; factors such as video length, upload date, subscribers and title are not included.
- **Descriptive analysis only.** The study describes the data and does not test hypotheses or make predictions.

15. Conclusion

This case study showed how Python can be used to create, check, analyse and visualize YouTube video data. NumPy and Pandas were used for the data and the statistics, and Matplotlib and Seaborn were used to draw ten different graphs. In the sample, the average video had 136,000 views, Tech videos had the highest average views (225,000) and Music videos had the lowest (70,000). Views, likes and comments increased together without exception.

The project also showed why different graphs suit different questions: a bar chart for comparing categories, a histogram for a distribution, a scatter plot and heatmap for relationships, and a count plot for group sizes. Since the dataset contains only ten illustrative records, the findings belong to this sample alone and should not be generalized to YouTube as a whole. The main value of the project is the practice it gives in the complete workflow of data analysis.

16. Future Scope

The following improvements are possible in the future. None of them has been implemented in the present project.

- Use a larger dataset, for example a trending-videos dataset downloaded from a public source, so that results are more reliable.
- Include more fields such as upload date, video length, number of subscribers and tags, which would allow time-based and more detailed analysis.
- Add data-cleaning steps for real data, such as handling missing values and removing duplicates when they actually occur.
- Build an interactive dashboard to explore the data.

- Apply statistical tests or predictive models to study the factors that influence engagement.

17. References

[1] McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.

[2] VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.

[3] Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

[5] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[6] The pandas development team. pandas documentation. https://pandas.pydata.org/docs/

[7] NumPy documentation. https://numpy.org/doc/

[8] Matplotlib documentation. https://matplotlib.org/stable/

[9] Seaborn documentation. https://seaborn.pydata.org/

[10] Python Software Foundation. Python documentation. https://docs.python.org/3/

18. Appendix A: Main Python Program (Analysis and Visualization)

This is the complete program that calculates the averages, category-wise averages and correlation matrix and draws all ten graphs shown in Section 12.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# YouTube data

categories = np.array([
    "Music", "Gaming", "Education", "Tech", "Comedy",
    "Music", "Gaming", "Education", "Tech", "Comedy"
])

views = np.array([
    50000, 80000, 120000, 200000, 150000,
    90000, 180000, 100000, 250000, 140000
])

likes = np.array([
    5000, 8000, 12000, 20000, 15000,
    9000, 18000, 10000, 25000, 14000
])

comments = np.array([
    500, 800, 1200, 2000, 1500,
    900, 1800, 1000, 2500, 1400
])

# Basic data analysis

print("YouTube Data Analysis")
print("Total Videos:", len(views))
print("Average Views:", np.mean(views))
print("Average Likes:", np.mean(likes))
print("Average Comments:", np.mean(comments))

highest = np.max(views)
print("Highest Views:", highest)

print("\nAverage Views by Category")

average_views = []

for category in np.unique(categories):
    category_views = views[categories == category]
    average = np.mean(category_views)
    average_views.append(average)
    print(category, ":", average)

# 1. Line Chart

plt.plot(views, marker="o")
plt.title("Views of Videos")
plt.xlabel("Video Number")
plt.ylabel("Views")
```

```
plt.show()

# 2. Bar Chart

plt.bar(np.unique(categories), average_views)
plt.title("Average Views by Category")
plt.xlabel("Category")
plt.ylabel("Average Views")
plt.xticks(rotation=20)
plt.show()

# 3. Histogram

plt.hist(comments, bins=5)
plt.title("Comments Distribution")
plt.xlabel("Comments")
plt.ylabel("Number of Videos")
plt.show()

# 4. Pie Chart

category_names, category_count = np.unique(
    categories, return_counts=True
)

plt.pie(
    category_count,
    labels=category_names,
    autopct="%1.1f%%"
)
plt.title("Videos by Category")
plt.show()

# 5. Scatter Plot

plt.scatter(views, likes)
plt.title("Views and Likes")
plt.xlabel("Views")
plt.ylabel("Likes")
plt.show()

# 6. Box Plot

sns.boxplot(x=categories, y=views)
plt.title("Views by Category")
plt.xlabel("Category")
plt.ylabel("Views")
plt.xticks(rotation=20)
plt.show()

# 7. Violin Plot

sns.violinplot(x=categories, y=views)
plt.title("Views Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Views")
plt.xticks(rotation=20)
plt.show()

# 8. Heatmap

data = np.array([views, likes, comments])
correlation = np.corrcoef(data)
```

```
sns.heatmap(
    correlation,
    annot=True,
    xticklabels=["Views", "Likes", "Comments"],
    yticklabels=["Views", "Likes", "Comments"]
)
plt.title("Correlation Heatmap")
plt.show()

# 9. Count Plot

sns.countplot(x=categories)
plt.title("Number of Videos in Each Category")
plt.xlabel("Category")
plt.ylabel("Number of Videos")
plt.xticks(rotation=20)
plt.show()

# 10. Pair Plot

data_for_pairplot = pd.DataFrame({
    "Views": views,
    "Likes": likes,
    "Comments": comments
})

sns.pairplot(data_for_pairplot)
plt.show()
```

19. Appendix B: Pandas Analysis Statements from the Project Report

The project report also contains the following analysis code, which performs the DataFrame creation, data-quality checks, median, standard deviation, grouping, sorting, filtering and correlation described in Sections 9 and 11. It is reproduced here as given in the report.

```
import numpy as np
import pandas as pd

# YouTube data
categories = np.array(["Music", "Gaming", "Education", "Tech", "Comedy",
                       "Music", "Gaming", "Education", "Tech", "Comedy"])
views = np.array([50000, 80000, 120000, 200000, 150000,
                  90000, 180000, 100000, 250000, 140000])
likes = np.array([5000, 8000, 12000, 20000, 15000,
                  9000, 18000, 10000, 25000, 14000])
comments = np.array([500, 800, 1200, 2000, 1500,
                     900, 1800, 1000, 2500, 1400])

# Create DataFrame
df = pd.DataFrame({"Category": categories, "Views": views,
                   "Likes": likes, "Comments": comments})
print("YouTube Data")
print(df)

# Data cleaning checks
print("\nMissing Values")
print(df.isnull().sum())
print("\nDuplicate Records")
print(df.duplicated().sum())
print("\nData Types")
print(df.dtypes)

# Statistical analysis
print("\nAverage Views:", np.mean(views))
print("Average Likes:", np.mean(likes))
print("Average Comments:", np.mean(comments))
print("\nMedian Views:", np.median(views))
print("Median Likes:", np.median(likes))
print("Median Comments:", np.median(comments))
print("\nStandard Deviation of Views:", np.std(views))
print("\nAverage Views by Category")
print(df.groupby("Category")["Views"].mean())
print("\nVideos Sorted by Views")
print(df.sort_values("Views", ascending=False))
print("\nVideos with More Than 150000 Views")
print(df[df["Views"] > 150000])
print("\nCorrelation")
print(df[["Views", "Likes", "Comments"]].corr())
```